

pg_dump and pg_restore — Logical Backup and Restore

Table of Contents

1. [Learning Objectives](#)
 2. [pg_dump Overview](#)
 3. [Backup Formats](#)
 4. [pg_dump Options and Flags](#)
 5. [pg_dumpall — Full Cluster Backup](#)
 6. [pg_restore — Restoring from Backup](#)
 7. [Selective Restore Patterns](#)
 8. [Parallel Backup and Restore](#)
 9. [Production Backup Scripts](#)
 10. [Common Mistakes](#)
 11. [Best Practices](#)
 12. [Performance Considerations](#)
 13. [Interview Questions](#)
 14. [Exercises and Solutions](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Use pg_dump for all backup formats (plain, custom, directory, tar)
 - Restore selectively: single table, single schema, exclude tables
 - Perform parallel backup and restore for large databases
 - Write production-grade backup scripts with monitoring
 - Use pg_dumpall for complete cluster backups
-

pg_dump Overview

`pg_dump` creates a consistent snapshot of a single PostgreSQL database. It connects to the database, starts a transaction with `REPEATABLE READ`, and exports data while the database continues to serve other connections.

`pg_dump` flow:

1. Connect to database
2. `SET TRANSACTION ISOLATION LEVEL SERIALIZABLE DEFERRABLE`
(for consistent snapshot across all tables)
3. Lock tables (briefly, for schema only in custom format)
4. Read schema: tables, views, functions, sequences, etc.
5. Dump data: `SELECT * FROM` each table
6. Output to file/stdout

Result: Consistent dump even for concurrent modifications

Backup Formats

Plain SQL Format (-F p)

```
# Plain SQL: human-readable, executable with psql
pg_dump -h localhost -U postgres -d mydb -F p -f mydb_backup.sql

# Compressed plain SQL
pg_dump -h localhost -U postgres -d mydb -F p | gzip > mydb_backup.sql.gz

# Restore plain SQL
psql -h restore_host -U postgres -d mydb_restored -f mydb_backup.sql
# Or from gzip:
gunzip -c mydb_backup.sql.gz | psql -h restore_host -U postgres -d mydb_restored
```

Custom Format (-F c) [RECOMMENDED]

```
# Custom format: compressed, supports parallel restore and selective restore
pg_dump -h localhost -U postgres -d mydb -F c -f mydb_backup.dump

# Compression level (0=none, 9=max, default=6)
pg_dump -F c -Z 9 -f mydb_backup.dump mydb

# View contents without restoring
pg_restore --list mydb_backup.dump

# Restore custom format
pg_restore -h restore_host -U postgres -d mydb_restored mydb_backup.dump
```

Directory Format (-F d)

```
# Directory format: one file per table, supports parallel dump AND restore
pg_dump -h localhost -U postgres -d mydb -F d -f mydb_backup_dir/

# With parallel jobs (e.g., 4 parallel dump jobs)
pg_dump -h localhost -U postgres -d mydb -F d -j 4 -f mydb_backup_dir/

# Directory structure:
# mydb_backup_dir/
#   toc.dat           (table of contents)
#   1234.dat.gz       (table data files, one per table)
#   ...

# Restore directory format (parallel)
pg_restore -h restore_host -U postgres -d mydb_restored -j 4 mydb_backup_dir/
```

Tar Format (-F t)

```
# Tar format: single file, like directory format but in tar archive
pg_dump -h localhost -U postgres -d mydb -F t -f mydb_backup.tar

# Restore tar format
pg_restore -h restore_host -U postgres -d mydb_restored mydb_backup.tar
```

Format Comparison

Format	Extension	Parallel?	Selective?	Compress?	Readable?
plain	.sql	No	No	External	Yes
custom	.dump	No dump/Yes restore	Yes	Built-in	No
directory	dir/	Yes both	Yes	Built-in	No
tar	.tar	No	Yes	External	No

Recommendation: Use custom (-F c) for most cases; directory (-F d) for very large databases where parallel dump is needed.

pg_dump Options and Flags

Selection Options

```
# Dump only schema (no data)
pg_dump -h localhost -U postgres -d mydb --schema-only -f schema.sql

# Dump only data (no schema)
pg_dump -h localhost -U postgres -d mydb --data-only -f data.sql

# Dump specific schema only
pg_dump -h localhost -U postgres -d mydb -n public -f public_schema.dump -F c

# Dump specific table only
pg_dump -h localhost -U postgres -d mydb -t orders -f orders.dump -F c

# Dump multiple tables
pg_dump -h localhost -U postgres -d mydb -t orders -t order_items -F c -f
orders_tables.dump

# Dump tables matching pattern
pg_dump -h localhost -U postgres -d mydb -t 'order*' -F c -f order_tables.dump

# Exclude a table from dump
pg_dump -h localhost -U postgres -d mydb -T audit_log -F c -f mydb_no_audit.dump

# Exclude tables matching pattern
pg_dump -h localhost -U postgres -d mydb -T 'temp_*' -F c -f mydb.dump
```

Data Handling Options

```
# Dump with no owner (useful when restoring to different environment)
pg_dump -h localhost -U postgres -d mydb -F c --no-owner -f mydb.dump

# Dump without privilege grants (GRANT/REVOKE)
pg_dump -h localhost -U postgres -d mydb -F c --no-privileges -f mydb.dump

# Dump with both options (most portable)
pg_dump -h localhost -U postgres -d mydb -F c --no-owner --no-privileges -f mydb.dump

# Use INSERT instead of COPY for data (slower but more portable)
pg_dump -h localhost -U postgres -d mydb --inserts -F p -f mydb_inserts.sql

# Include column names in INSERT statements
pg_dump -h localhost -U postgres -d mydb --column-inserts -F p -f mydb.sql

# Add IF EXISTS to DROP statements
pg_dump -h localhost -U postgres -d mydb --if-exists -F c -f mydb.dump
```

Connection Options

```
# Full connection specification
pg_dump \
  --host=db.example.com \
  --port=5432 \
  --username=postgres \
  --dbname=mydb \
  --no-password \           # Use .pgpass or PGPASSWORD env var
  --format=custom \
  --file=mydb.dump

# Using connection string
pg_dump "postgresql://postgres@db.example.com:5432/mydb?sslmode=verify-full" \
  -F c -f mydb.dump

# Dump from standby (reduce primary load)
pg_dump -h standby.example.com -U postgres -d mydb -F c -f mydb.dump
```

pg_dumpall — Full Cluster Backup

`pg_dumpall` backs up ALL databases in a PostgreSQL cluster plus global objects (roles, tablespaces).

```
# Full cluster backup
pg_dumpall -h localhost -U postgres -f cluster_backup.sql

# Only global objects (roles, tablespaces) – no database data
```

```
pg_dumpall -h localhost -U postgres --globals-only -f globals.sql

# Only roles (for creating users in new cluster)
pg_dumpall -h localhost -U postgres --roles-only -f roles.sql

# Only tablespaces
pg_dumpall -h localhost -U postgres --tablespaces-only -f tablespaces.sql

# Restore cluster backup
psql -h new_host -U postgres -f cluster_backup.sql

# Compressed
pg_dumpall -h localhost -U postgres | gzip > cluster_backup.sql.gz
gunzip -c cluster_backup.sql.gz | psql -h new_host -U postgres
```

pg_restore — Restoring from Backup

```
# Basic restore (database must exist)
pg_restore -h localhost -U postgres -d mydb mydb.dump

# Create database and restore
createdb -h localhost -U postgres mydb_restored
pg_restore -h localhost -U postgres -d mydb_restored mydb.dump

# Restore with verbose output
pg_restore -h localhost -U postgres -d mydb_restored -v mydb.dump

# Create db, restore in one step
pg_restore -h localhost -U postgres -C mydb.dump
# -C creates the database as named in the dump

# Drop existing objects before restore (clean restore)
pg_restore -h localhost -U postgres -d mydb_restored --clean mydb.dump

# Restore with IF EXISTS (combined with --clean)
pg_restore -h localhost -U postgres -d mydb_restored --clean --if-exists mydb.dump

# No owner (don't try to set ownership)
pg_restore -h localhost -U postgres -d mydb_restored --no-owner mydb.dump
```

Selective Restore Patterns

Restore Single Table

```
# Method 1: Using -t flag
pg_restore -h localhost -U postgres -d mydb -t orders mydb.dump

# Method 2: Using list file
```

```
# First, get the list of objects in the dump
pg_restore --list mydb.dump > restore_list.txt

# The list looks like:
# 123; 1259 16384 TABLE public orders postgres
# 456; 1259 16385 TABLE public customers postgres
# ...

# Edit to keep only the tables you want, then restore
grep "TABLE public orders" restore_list.txt > orders_only.txt
pg_restore -h localhost -U postgres -d mydb -L orders_only.txt mydb.dump
```

Restore Single Schema

```
pg_restore -h localhost -U postgres -d mydb -n analytics mydb.dump
```

Restore Data Only (Schema Exists)

```
pg_restore -h localhost -U postgres -d mydb --data-only mydb.dump
```

Restore Schema Only

```
pg_restore -h localhost -U postgres -d mydb --schema-only mydb.dump
```

Restore Specific Object Types

```
# First list all objects
pg_restore --list mydb.dump

# Output example:
# ;
# ; Archive created at 2024-01-15 10:30:00 UTC
# ; dbname: mydb
# ;
# 2; 1262 16384 DATABASE - mydb postgres
# 5; 0 0 ENCODING - ENCODING -
# 6; 0 0 STDSTRINGS - STDSTRINGS -
# 8; 1259 16387 TABLE public orders postgres
# 9; 0 16387 TABLE DATA public orders postgres
# 10; 0 0 SEQUENCE SET public orders_id_seq postgres
# ...

# Restore only table definitions (no data)
pg_restore --list mydb.dump | grep "TABLE " | grep -v "TABLE DATA" > schema_only.lst
pg_restore -h localhost -U postgres -d mydb -L schema_only.lst mydb.dump
```

Parallel Backup and Restore

```
# Parallel dump (directory format only!)
# -j N: use N parallel worker processes
pg_dump -h localhost -U postgres -d mydb \
  -F d \          # Directory format (required for parallel)
  -j 4 \          # 4 parallel jobs
  -f /backup/mydb_dir/

# Monitor parallel dump progress
ls -la /backup/mydb_dir/ | wc -l # Increasing file count

# Parallel restore (directory or custom format)
pg_restore -h localhost -U postgres -d mydb_restored \
  -j 4 \          # 4 parallel restore workers
  /backup/mydb_dir/

# For custom format with parallel restore:
pg_restore -h localhost -U postgres -d mydb_restored \
  -j 4 \
  mydb_backup.dump # Custom format also supports parallel restore!

# Rule of thumb: -j = number of CPU cores / 2
# More is not always better: I/O becomes the bottleneck
```

Production Backup Scripts

Daily Backup Script

```
#!/bin/bash
# pg_dump_backup.sh – Production backup script

set -euo pipefail

# Configuration
PGHOST="${PGHOST:-localhost}"
PGPORT="${PGPORT:-5432}"
PGUSER="${PGUSER:-postgres}"
PGDATABASE="${PGDATABASE:-mydb}"
BACKUP_DIR="/var/backups/postgresql"
RETENTION_DAYS=30
TIMESTAMP=$(date +%Y-%m-%d_%H%M%S)
LOG_FILE="/var/log/postgresql/backup_${TIMESTAMP}.log"

# Functions
log() {
  echo "$(date '+%Y-%m-%d %H:%M:%S') $1" | tee -a "$LOG_FILE"
}
```

```

error_exit() {
    log "ERROR: $1"
    # Send alert
    # echo "$1" | mail -s "PostgreSQL Backup FAILED" dba@company.com
    exit 1
}

# Create backup directory
mkdir -p "$BACKUP_DIR"

BACKUP_FILE="${BACKUP_DIR}/${PGDATABASE}_${TIMESTAMP}.dump"

log "Starting backup of ${PGDATABASE} on ${PGHOST}"
log "Backup file: ${BACKUP_FILE}"

# Run backup
START_TIME=$(date +%s)

pg_dump \
    --host="${PGHOST}" \
    --port="${PGPORT}" \
    --username="${PGUSER}" \
    --format=custom \
    --compress=6 \
    --no-password \
    --verbose \
    --file="${BACKUP_FILE}" \
    "${PGDATABASE}" >> "$LOG_FILE" 2>&1 || error_exit "pg_dump failed"

END_TIME=$(date +%s)
DURATION=$((END_TIME - START_TIME))
BACKUP_SIZE=$(du -sh "$BACKUP_FILE" | cut -f1)

log "Backup completed in ${DURATION}s, size: ${BACKUP_SIZE}"

# Verify backup is non-empty
if [ ! -s "$BACKUP_FILE" ]; then
    error_exit "Backup file is empty!"
fi

# Verify backup can be listed (basic integrity check)
pg_restore --list "$BACKUP_FILE" > /dev/null || error_exit "Backup integrity check failed"
log "Backup integrity check passed"

# Upload to S3 (optional)
# aws s3 cp "$BACKUP_FILE" "s3://my-pg-backups/${PGDATABASE}/${TIMESTAMP}.dump"
# log "Uploaded to S3"

# Cleanup old backups
log "Cleaning up backups older than ${RETENTION_DAYS} days"

```



```
find "$BACKUP_DIR" -name "${PGDATABASE}*.dump" -mtime +${RETENTION_DAYS} -delete
REMAINING=$(find "$BACKUP_DIR" -name "${PGDATABASE}*.dump" | wc -l)
log "Remaining backup count: ${REMAINING}"

log "Backup script completed successfully"
```

Verification Script

```
#!/bin/bash
# verify_backup.sh – Restore backup to test and verify

BACKUP_FILE=$1
TEST_DB="verify_test_$(date +%s)"
PGHOST_RESTORE="${PGHOST_RESTORE:-localhost}"

echo "Verifying backup: $BACKUP_FILE"

# Create test database
createdb -h "$PGHOST_RESTORE" -U postgres "$TEST_DB" || exit 1

# Restore to test database
pg_restore -h "$PGHOST_RESTORE" -U postgres -d "$TEST_DB" "$BACKUP_FILE" 2>/dev/null
RESTORE_EXIT=$?

if [ $RESTORE_EXIT -eq 0 ]; then
    # Run basic checks
    TABLE_COUNT=$(psql -h "$PGHOST_RESTORE" -U postgres -d "$TEST_DB" -t -c \
        "SELECT COUNT(*) FROM information_schema.tables WHERE table_schema =
        'public';")
    echo "Restore successful! Tables: $TABLE_COUNT"
else
    echo "Restore had warnings (exit code $RESTORE_EXIT) – check for errors"
fi

# Cleanup
dropdb -h "$PGHOST_RESTORE" -U postgres "$TEST_DB"
echo "Test database dropped"
```

Common Mistakes

1. **Using plain format for large databases** — slow restore, no selective restore, no compression
2. **Not using `--no-owner`** when restoring to different user — role doesn't exist errors
3. **Restoring to non-empty database** without `--clean` — duplicate object errors
4. **Forgetting to backup globals** (roles/tablespaces) — need `pg_dumpall --globals-only`
5. **Not testing restores** — backup file corruption discovered during actual recovery
6. **PGPASSWORD in ps aux** — use .pgpass instead
7. **Not monitoring backup size changes** — dramatic size drops indicate data loss
8. **Running pg_dump on primary** for large DBs — use standby to avoid performance impact
9. **No compression** — wasting storage for large databases

10. `-j` **parallel dump without** `-F d` — silently ignored, not parallel
-

Best Practices

1. Use **custom format** (`-F c`) as default — compressed, selective restore, parallel restore
 2. Use **directory format** (`-F d`) for large databases needing parallel dump
 3. Always include `--no-owner` for portability
 4. Back up **global objects separately** (`pg_dumpall --globals-only`)
 5. Run backups from **standby** to reduce primary load
 6. **Verify every backup** — at minimum run `pg_restore --list`
 7. **Test full restores monthly** — don't just verify, actually restore
 8. Set **retention policy** and automate cleanup
 9. **Monitor backup duration** — significant increases may indicate issues
 10. **Store backups offsite** — different location than production data
-

Performance Considerations

```
# Estimate backup time for large tables
# Rule of thumb: pg_dump processes ~500MB-2GB per minute (disk I/O bound)

# Speed up backup:
# 1. Use directory format with parallel jobs
pg_dump -F d -j 8 -f /backup/mydb/

# 2. Reduce compression (faster backup, larger file)
pg_dump -F c -Z 1 -f mydb_fast.dump # Level 1 compression

# 3. Dump from replica (doesn't impact primary)
pg_dump -h replica.internal ...

# Speed up restore:
# 1. Parallel restore
pg_restore -j 8 -d mydb /backup/mydb/

# 2. Disable indexes during restore, rebuild after
pg_restore --no-data-for-failed-tables ...
# Or: disable triggers temporarily, load data, rebuild indexes
```

Interview Questions

Q1: What is the difference between `pg_dump` and `pg_dumpall`?

A: `pg_dump` backs up a single database (schema + data). `pg_dumpall` backs up ALL databases in a PostgreSQL cluster plus global objects: roles (users/groups), tablespaces. `pg_dump` cannot backup roles because they're cluster-level objects, not database-level. For a complete backup, you need both: `pg_dump` for each database and `pg_dumpall --globals-only` for roles/tablespaces.

Q2: What formats does `pg_dump` support and when do you use each?

A: Plain (`.sql`): human-readable SQL, restores with `psql`; good for small DBs, human review. Custom (`.dump`): compressed, supports selective restore and parallel `pg_restore`; best default for most databases. Directory: one file per object, supports parallel dump AND restore; best for large databases. Tar: like directory but in tar file; less common.

Q3: How do you restore a single table from a `pg_dump` backup?

A: Use `pg_restore -t tablename`. For custom format: `pg_restore -h host -U user -d dbname -t orders mydb.dump`. For directory format: same. For plain SQL: you'd need to manually extract the relevant INSERT/COPY statements, or use `--list` to identify the section and `--use-list` to restore only that section.

Q4: What is the `--no-owner` option for?

A: Without `--no-owner`, `pg_restore` tries to set the database objects' ownership to match the original dump. If those roles don't exist on the restore server, it fails with "role X does not exist." `--no-owner` skips the ownership assignments, making objects owned by the restoring user. Essential when restoring to a different environment with different role names.

Q5: How does parallel `pg_dump` work?

A: Parallel dump (`-j N`) only works with directory format (`-F d`). It opens N connections to PostgreSQL and dumps different tables simultaneously. Each table is written to a separate file. The master process reads the schema; worker processes read table data. The `toc.dat` file records what was dumped. Parallel dump speeds up export proportional to I/O parallelism.

Q6: What happens if `pg_dump` fails midway?

A: The output file will be incomplete. For custom/directory formats, `pg_restore` will fail when it encounters the truncated/missing data. For plain format, the SQL file will be incomplete and restore will fail partway through. Always check `pg_dump` exit code (`$?`), and verify the backup with `pg_restore --list`.

Q7: Can `pg_dump` capture changes made during the dump?

A: `pg_dump` takes a consistent snapshot at the start of the export using a repeatable read transaction. Changes made by other transactions during the dump are NOT captured — the dump reflects the database state at dump start time. This is correct behavior for a backup; it's why `pg_dump` is suitable for backups of live databases.

Exercises and Solutions

Exercise 1: Selective Restore Workflow

```
# Scenario: Accidentally dropped the orders table. Restore only orders from
yesterday's backup.

# Step 1: List backup contents
pg_restore --list /backup/mydb_20240115_020000.dump | grep orders

# Output:
# 8; 1259 16387 TABLE public orders postgres
# 9; 0 16387 TABLE DATA public orders postgres
# 10; 0 0 SEQUENCE SET public orders_id_seq postgres
```

```
# Step 2: Create list file for orders
pg_restore --list /backup/mydb_20240115_020000.dump | \
    grep -E "(TABLE public orders|TABLE DATA public orders|SEQUENCE SET public
orders)" \
    > orders_restore.lst

# Step 3: Restore only orders
pg_restore -h localhost -U postgres -d mydb -L orders_restore.lst \
    /backup/mydb_20240115_020000.dump

echo "Orders table restored!"
```

Exercise 2: Compare backup sizes across formats

```
#!/bin/bash
for FORMAT in p c d t; do
    case $FORMAT in
        p) EXT=sql;;
        c) EXT=dump;;
        d) EXT=dir;;
        t) EXT=tar;;
    esac

    if [ $FORMAT = 'd' ]; then
        pg_dump -U postgres -d mydb -F $FORMAT -f /tmp/test_backup.$EXT/
        SIZE=$(du -sh /tmp/test_backup.$EXT/ | cut -f1)
    else
        pg_dump -U postgres -d mydb -F $FORMAT -f /tmp/test_backup.$EXT
        SIZE=$(du -sh /tmp/test_backup.$EXT | cut -f1)
    fi

    echo "Format $FORMAT: $SIZE"
done
```

Cross-References

- [01_backup_strategies.md](#) — When to use pg_dump
- [03_pg_basebackup.md](#) — Physical backup alternative
- [06_pgbackrest.md](#) — Production backup tool
- [07_disaster_recovery.md](#) — Using pg_dump in DR scenarios